
Learning to Push: Imitation Learning and RL on PushT

Department of Mechanical Engineering
2026.10.07 (Wed)
Joo Yong Sim

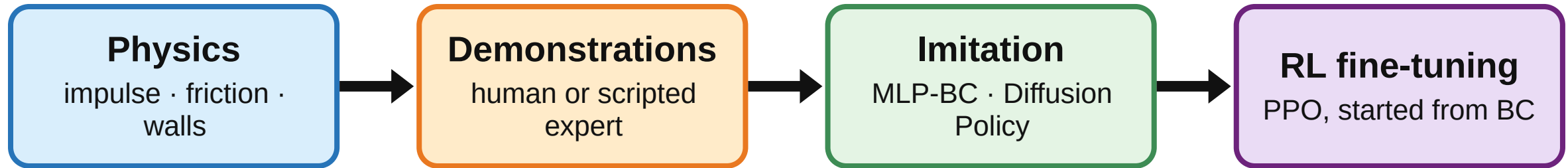


Hands-on tutorial: `pusht_imitation.py` · `pusht_rl.py` · `pusht_rl_realtime.py` · `pusht_rl_eval.py`

Lecture Goals: From Physics to Policy



The PushT simulator from the dynamics lecture becomes a training ground



Three questions answered with experiments in this lecture

Q1 — Data : how many demonstrations do we need to collect?

Q2 — Model : which network should learn from them — a regressor or a generative model?

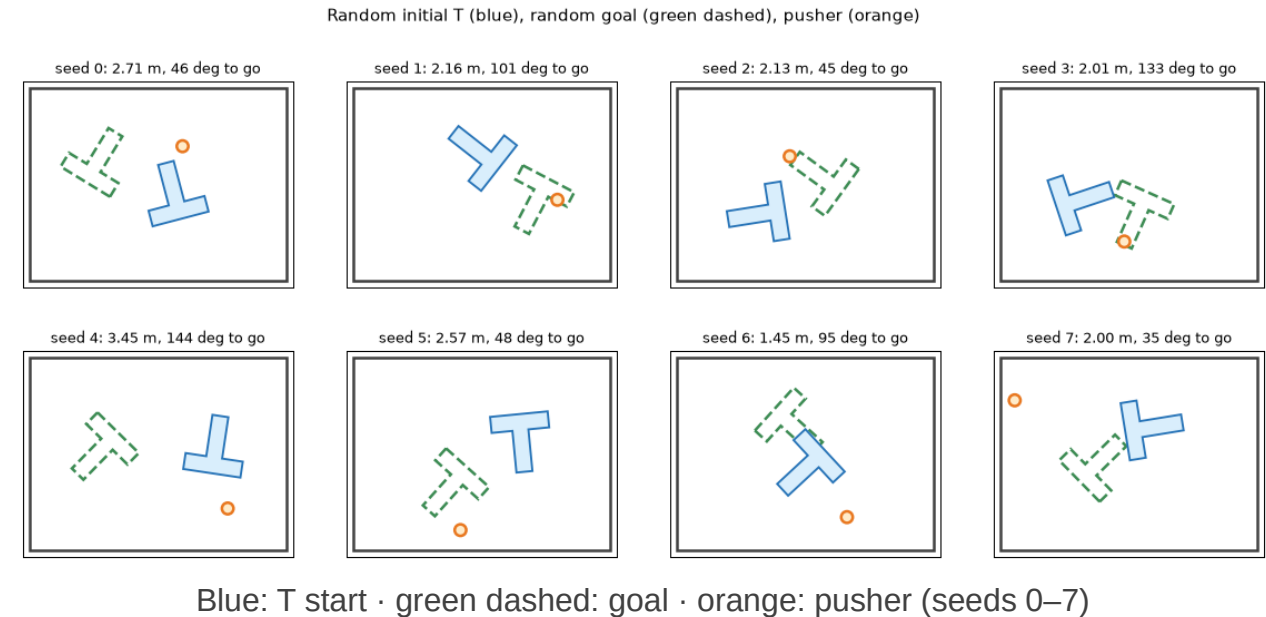
Q3 — Beyond imitation : can reinforcement learning do better, from scratch or starting from the imitation policy?

The PushT Task: Random Start, Random Goal



One seed fixes the initial T pose, the goal pose and the pusher position

What	Distribution
T start pose	
Goal pose	
Pusher	
	Success



Seeds : demos use 100000+ (expert) and 200000+ (human); **evaluation uses 0, 1, 2, ...** — tasks never seen in training.

Control Loop and Demonstration Data



The policy moves the pusher's target at 10 Hz; the physics runs 20 substeps in between

State (8 numbers — what a human sees)

One episode, saved to .npz

Action

A PD controller pulls the pusher toward the target;
contact with the T is the impulse model of the dynamics
lecture.

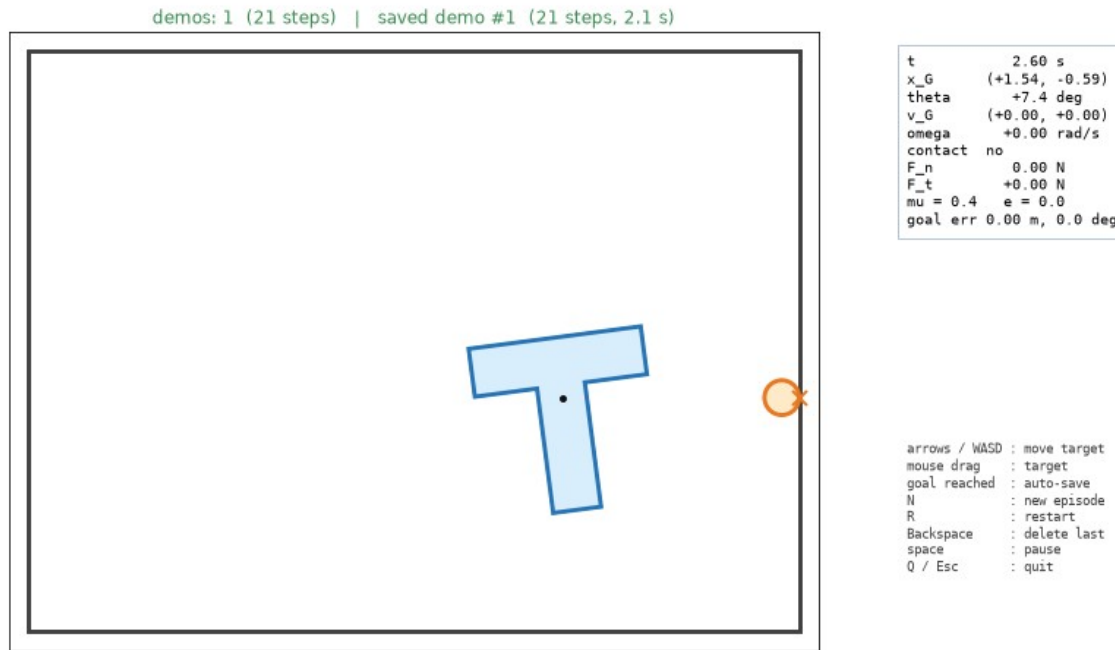
Array	Shape	Meaning
states	(N, 8)	
actions	(N, 2)	
episode_ends	(E,)	where each episode ends
seeds	(E,)	re-creates the start

Only successful episodes are saved : imitating a failure teaches failure (filtered behavior cloning). Same format as the Diffusion Policy / LeRobot PushT data.

Collecting Human Demonstrations



python pusht_imitation.py collect — the keyboard simulator with a recorder added



Recorder window: arrows / WASD or mouse drag move the target; reaching the goal auto-saves

Three rules for clean data

Idle start trimmed : time spent thinking before the first move is not recorded, so the policy never learns to stand still.

Physics fixed : friction / restitution switches are off — same state, same consequence.

Keys : N new task · R restart · Backspace delete last · Space pause

Cost : one human demo takes 15–30 s, so 200 demos take **1.5–2 hours** → we also need an automatic demonstrator.

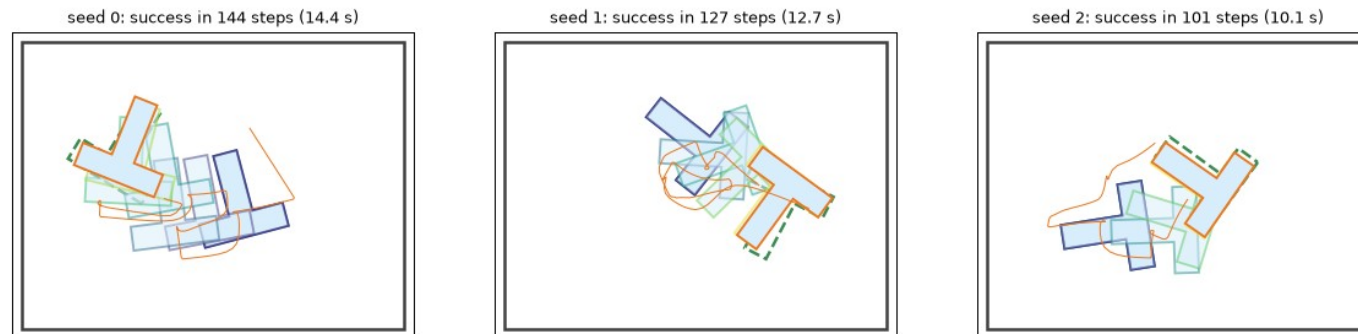
Scripted Expert: Impulse–Momentum as a Policy



A "virtual human" that chooses where to push with the lecture's impulse equations

(2) Approach : go to a standoff point; orbit around the T if the straight path is blocked.

Scripted expert (orange line = pusher path, colour = time)



Expert rollouts (orange: pusher path, colour: time) — success $\approx$ 88–91 % on random tasks

How Many Demonstrations? Reference Points



Published PushT and manipulation work uses 50–200 demonstrations per task

Source	Task	Demonstrations
Diffusion Policy (Chi et al., 2023)	PushT, fixed goal , random start	200 human ($\approx$ 25k steps)
LeRobot pusht dataset	same task	206 episodes, 25,650 frames
ACT / ALOHA (Zhao et al., 2023)	real bimanual robot	50 per task

Suggested plan for human demonstrations (15–30 s each)

Stage	Demos	Time	Purpose
Pipeline check	10–20	10 min	save / load / train runs, loss goes down
First results	50–100	30–60 min	successes start; study the failures
Usable policy	$\geq$ 200	1.5–2 h	Diffusion Policy paper scale

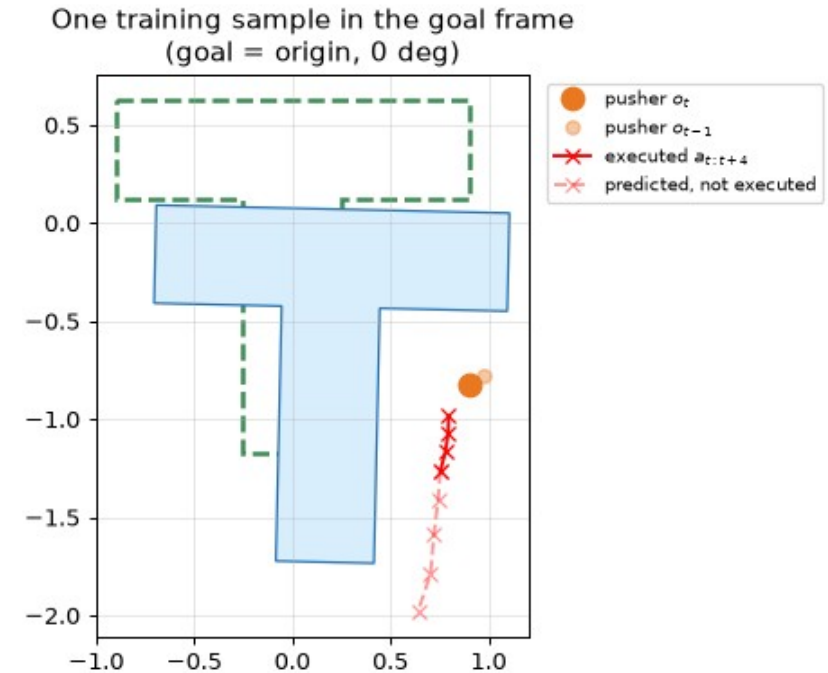
More important than the count : spread-out initial conditions, **a consistent strategy**, and discarded failures. Our goal is random too — harder than the original PushT.

Goal Frame: Relative Motion Removes the Random Goal



Ride along with the goal — the same idea as relative motion in rigid-body kinematics

Action representation (goal frame)	MLP-BC	Diffusion
absolute target position	26 %	20 %
offset from the current pusher	36 %	46 %



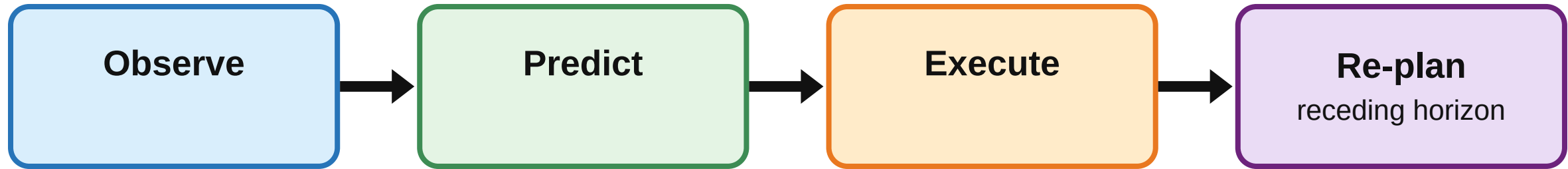
One training sample: goal at the origin, 0°

Without the goal frame : 0 % success for both models — a good representation saves data.

Observation History and Action Chunks



See two frames, plan eight actions, execute four, then re-plan



Two frames : the difference gives velocity — is the T still sliding?

Chunks : one-step predictions change every 0.1 s and jitter; a chunk commits to a motion for a moment.

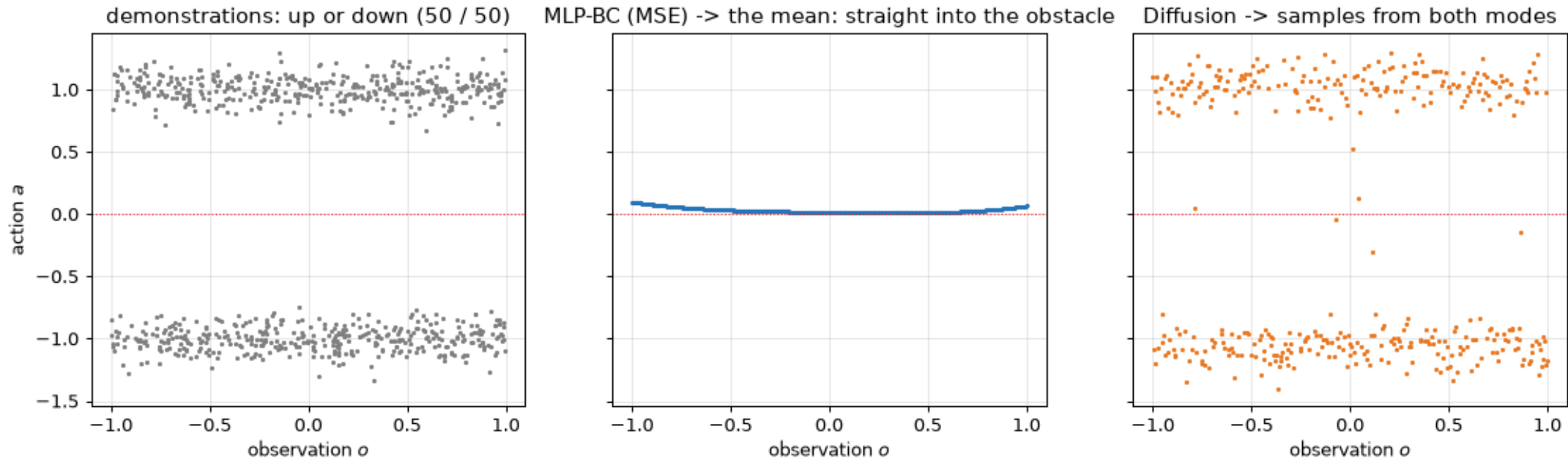
Split by episode : neighbouring steps are nearly identical — a step-wise split leaks data into validation.

Trade-off : 4 steps run open-loop, so an unexpected slide is corrected only at the next re-plan.

Which Model? The Multimodality Problem



MSE learns the average action — the average of two good actions can be bad



In PushT : "go around the left or the right side of the T?" — MLP-BC hesitates in between; human demos are even more multimodal.

Policy Model Candidates



Model	Output	Multimodal	Notes
MLP-BC (MSE)	one point	no (mean)	fastest, simplest — baseline
MLP-BC + action chunk		no	temporally consistent
GMM / MDN		partly	
VQ-BeT	discrete code + offset	yes	Transformer
ACT (CVAE + Transformer)	chunk	yes	real robot, 50 demos
Diffusion Policy	chunk, sampled	yes	standard PushT baseline, stable training

Our choice : MLP-BC as the baseline, Diffusion Policy as the main model — **same MLP trunk** (3 layers, width 512, Mish).

So the only difference is : learning the mean vs. learning the distribution.

Diffusion Policy (DDPM) in Three Steps



Learn to remove noise from action chunks; at run time, denoise noise into actions

2. Training: predict the noise

Training Settings Found by Experiment



Diffusion needs more capacity or more steps than MLP-BC before it pays off

Setting (180 expert demos, unseen tasks)	MLP-BC	Diffusion
	32 %	8 %
	36–46 %	18–42 %
	—	64 %
	38–59 %	54–63 %
	—	69 %

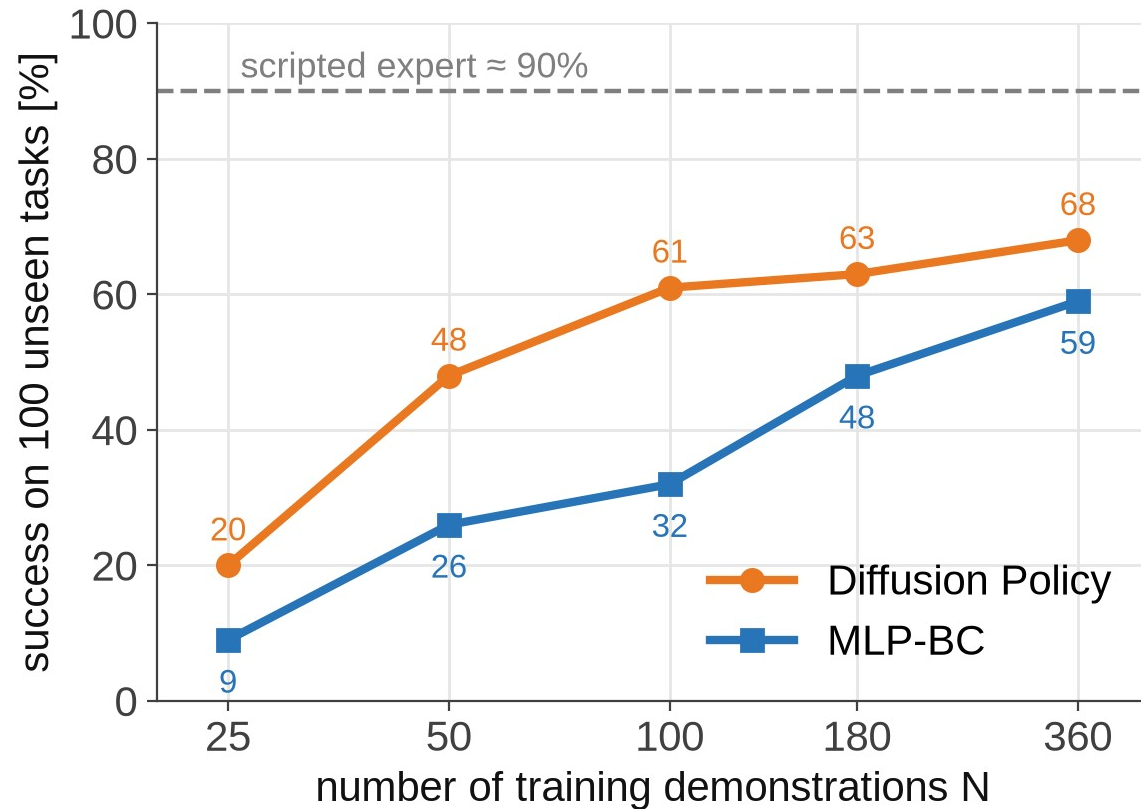
Batch 256, AdamW, cosine lr decay; about 1 min per model on GPU, 4 min on CPU.

Caution : re-running the same setting moves success by ± 10 %; validation loss does not predict success (0.116 $\rightarrow$ 0.132 while 62 % $\rightarrow$ 69 %).

How Many Demonstrations? Measured



Diffusion reaches most of its performance at about 100 demonstrations



Experiment

Reading

MLP-BC needs about 3× more demos for the same success.

Human demos : aim for **at least 100, ideally 200**.

Reproduce : tools/imitation_experiments/full/run_scale512.sh (10 jobs, about 15 min)

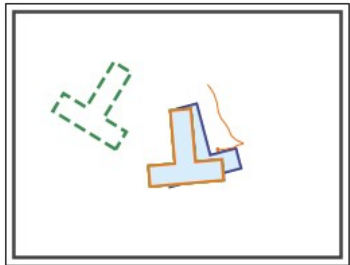
Closed-Loop Rollouts and Failure Modes



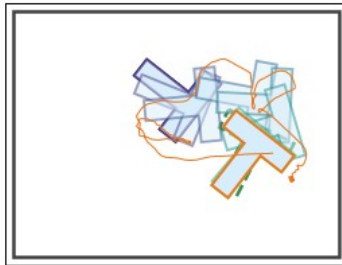
Both learned policies stay below the expert that taught them

MLP-BC rollouts on unseen seeds

seed 0: fail (300 steps)



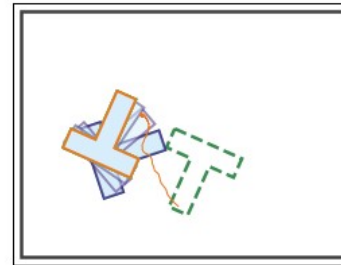
seed 1: fail (300 steps)



seed 2: success (153 steps)



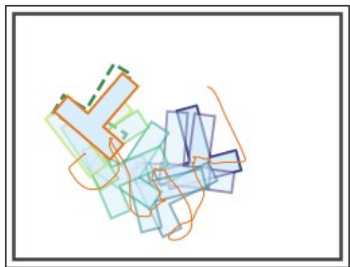
seed 3: fail (300 steps)



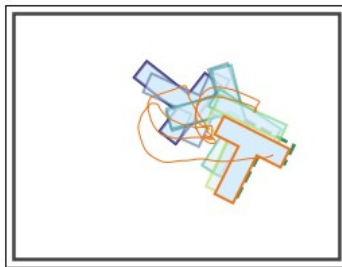
MLP-BC rollouts on unseen seeds

Diffusion rollouts on unseen seeds

seed 0: success (225 steps)



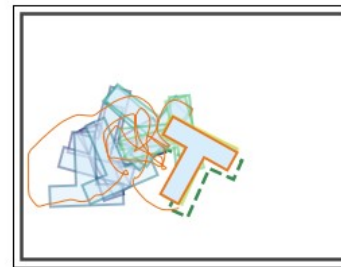
seed 1: success (198 steps)



seed 2: fail (300 steps)



seed 3: fail (300 steps)



Diffusion Policy rollouts on the same seeds

MLP-BC : stalls or jitters beside the T — it averages between strategies.

Diffusion : commits to one strategy; fails at the final 0.15 m / 10° alignment or after one big mistake.

Both : **covariate shift** — their own errors lead to states never seen in the demos.

Default run (180 demos) : expert 88 % · MLP-BC 38–59 % · Diffusion 54–63 %. Remedy: DAgger — record human corrections.

From Imitation to Reinforcement Learning



RL needs no demonstrations — only a reward and a great deal of trial and error

Experiment	Starting point	Result (50 unseen tasks, no exploration noise)
A. RL from scratch	random network	16.4M steps (2.5 h): 0 % success
B. BC → RL	MLP-BC policy	42 % → 74 % (3M) → 80 % (6M, 80 min)
Scripted expert	hand-written rules	88 %

Why A fails : success needs a long sequence — choose a contact, go around, push, align. Random motion almost never completes it, so the success bonus is **never observed** (the exploration problem).

Why B works : imitation already succeeds about half the time; RL only has to polish behaviour that works.

Reward Design



Turn "how far is the task from done" into one number, and reward every step of progress

Remaining cost (1 rad $\approx$ 0.3 m)

Reward per step

Side effect : the time penalty makes RL **finish faster** than the expert, which has no such pressure.

PPO: Small, Clipped Policy Updates



Collect 8,192 steps, estimate advantages, update for 10 epochs — repeat

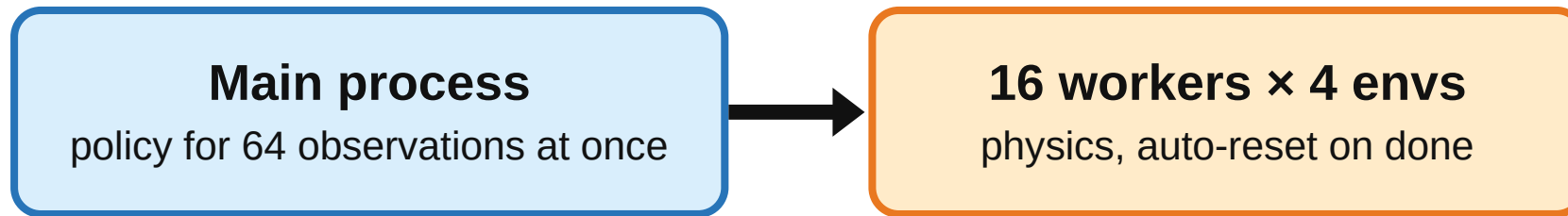
	A. scratch	B. BC → RL
Actor	$2 \times 256 \tanh$	the MLP-BC net
	0.6	0.03 m
Actor lr		
Critic	new	new, 10 warm-up updates

For B : clipping, a small learning rate and small noise **protect the working BC policy** from being destroyed early on.

Parallel Simulation



Pure-Python physics is slow, so 64 environments run at once on 16 CPU processes



Benchmark (idle CPU, 20 cores)

Steps per PPO iteration	$64 \times 128 = 8,192$
Rollout (physics 3.09 s)	3.79 s
PPO update	1.91 s (33 %)
Env steps / s	2,159 rollout · 1,437 total

Training wall-clock (from logs)

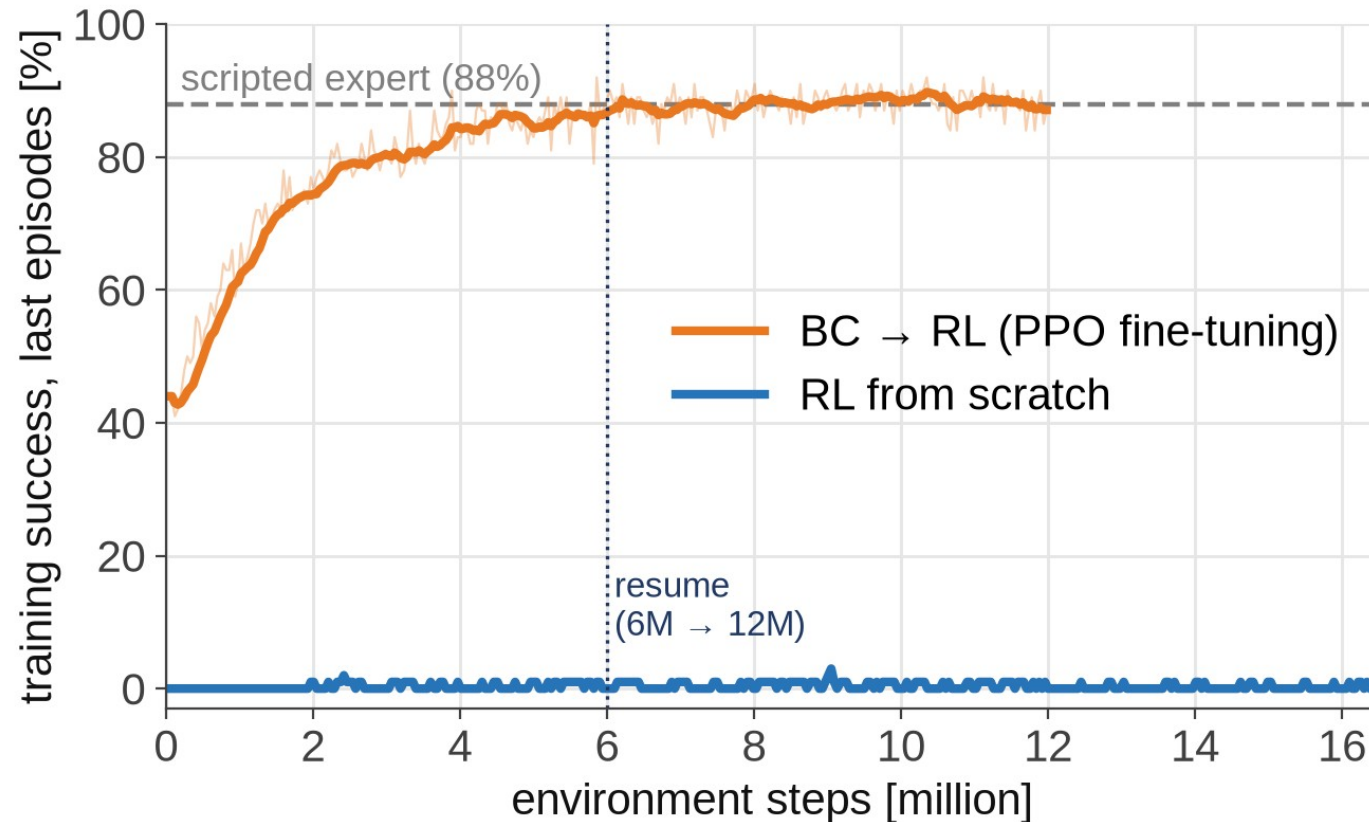
Run	Steps	Time
RL from scratch	16.4M	147 min
BC+PPO	0 → 6M	80 min
BC+PPO resumed	6 → 12M	70 min

Backend : CPU multiprocessing (fork + Pipe); the network also runs on the CPU. One environment alone would be **$\approx 64\times$ slower**.

RL from Scratch vs BC → RL



Starting from imitation turns an unsolvable exploration problem into fine-tuning



Training success with exploration noise (tutorial/checkpoints/rl/*/train_log*.txt)

From scratch : return improved $-14 \rightarrow -2.4$, the T moves toward the goal, but **0 % success**; exploration noise collapsed $0.6 \rightarrow 0.03$.

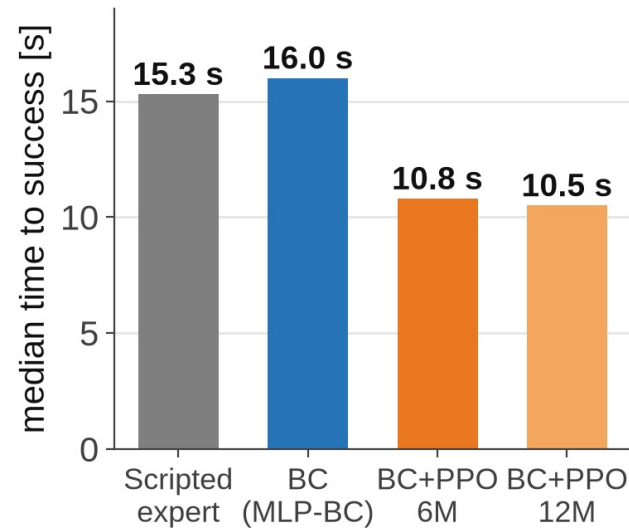
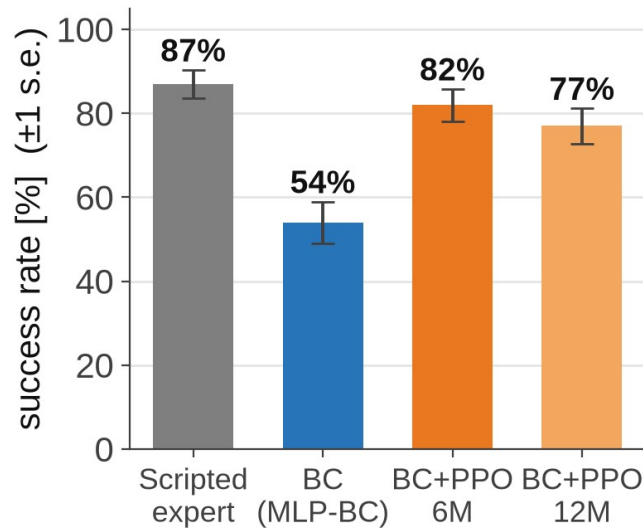
BC → RL : $0.44 \rightarrow 0.88$ in 6M steps, then a plateau at 0.87–0.90.

Lesson : demonstrations solve exploration; RL then optimizes what imitation cannot — the reward itself.

Paired Evaluation on Seeds 0–99



Same 100 unseen tasks, same start, 300 steps, deterministic actions for every policy



pusht_rl_eval.py → tutorial/eval_results/paired_*.csv / .json

Scripted vs BC+PPO	6M	12M
both succeed	74	70
only scripted	13	17
only BC+PPO	8	7
both fail	5	6
	0.38	0.064
median steps*	136 → 107	139 → 104

* on seeds both solved: scripted → BC+PPO

Result : success is **statistically tied with the expert**, and BC+PPO is about 3–4 s faster on the same tasks.

Why BC+PPO Stops Near the Expert



More training made the policy faster, not more successful

Measured

Within noise : at 82 %, one standard error is about ± 4 points with 100 tasks.

6M $\rightarrow$ **12M** : success 82 % $\rightarrow$ 77 %, mean time 12.0 s $\rightarrow$ 11.1 s.

Best checkpoint : 6M, not the newest 12M.

Likely structural limits (not yet tested)

Less information : 12 inputs vs. the expert's full state and memory.

Reward $\neq$ success test : angle weighted 0.3; success needs 0.15 m and 10° .

One step at a time : BC drops 54 % $\rightarrow$ 34 % without chunks.

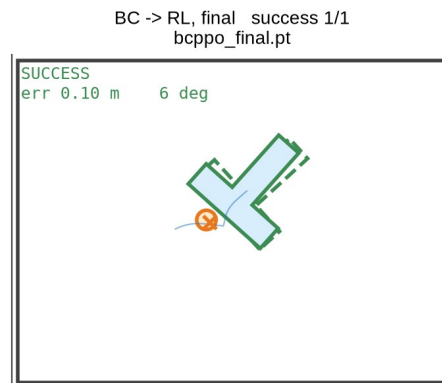
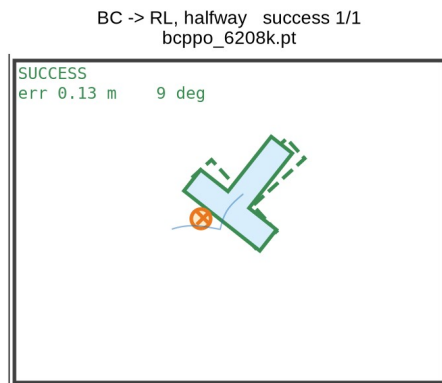
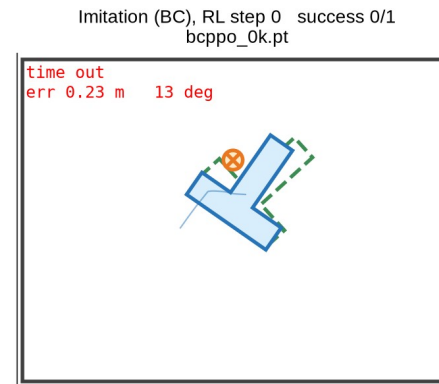
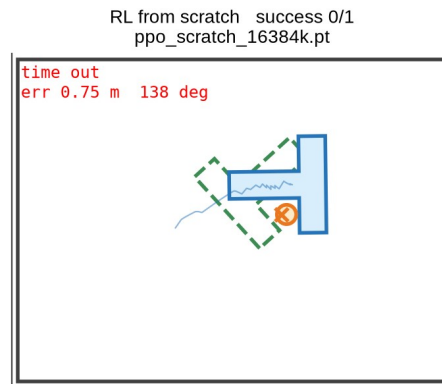
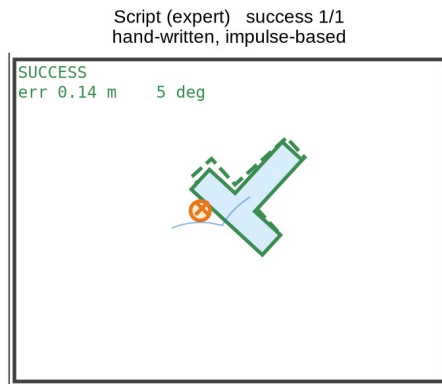
Not a ceiling : RL optimizes the reward, not imitation — align the reward with the success test to push further.

Real-Time Demo: Five Policies, One Task



python pusht_rl_realtime.py — every panel replays the same random task live

pusht_rl_realtime.py — all five policies on the same unseen task (seed 30)



Seed 30: script 8.7 s · BC → RL 6.4 / 6.2 s · BC and RL-from-scratch time out

Run

```
cd tutorial
python pusht_rl_realtime.py
python pusht_rl_realtime.py script
python pusht_rl.py eval
python pusht_rl_eval.py
```

Keys

Space pause · N next task · + / – speed ·
Q quit



Physics gives the data, imitation gives the start, RL gives the polish

- - Goal frame (relative motion) and action offsets matter as much as the model
- Data: about 100 demos for Diffusion Policy; collect 100–200 human demos
- - Measured: Diffusion 20–68 % vs MLP-BC 9–59 % (25–360 demos)
- RL from scratch: 0 % after 16M steps — the exploration problem
 - BC → PPO: 54 % → 82 % on seeds 0–99, tied with the expert (87 %) and 3–4 s faster



Exercises with the tutorial code

- **Human data** : collect 50, then 100 demos — does the Diffusion vs MLP-BC gap grow?
- **No goal frame** : train in world coordinates — how much success is lost?
- **Chunks** : vary the prediction / execution horizons (1, 4, 8, 16)
- **History** : one observation frame — which mistakes appear without velocity?
- **Dagger** : record your corrections in the viewer and retrain
- **Robustness** : evaluate with friction 0.2 instead of 0.4 — think domain randomization
- **RL** : put the 0.15 m / 10° test into the reward — does BC+PPO pass the expert?